

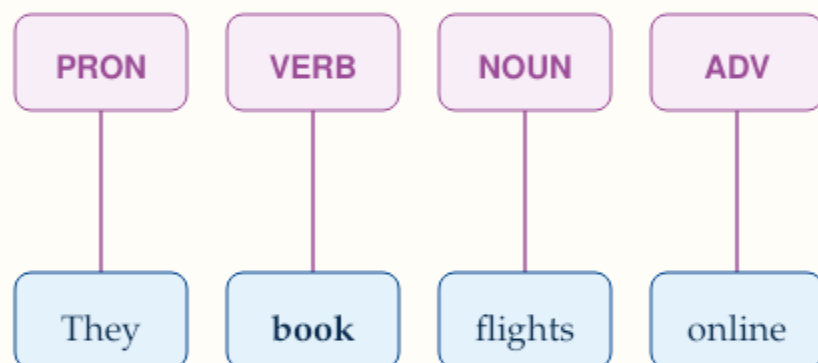
When Inputs and Outputs Do Not Align

Translation can change both sequence length and word order.

FROM ALIGNED READOUTS TO VARIABLE-LENGTH GENERATION

Aligned sequence labelling

Each input position has one output position.



The readout attaches to the recurrent state at the same position.

One shared input–output timeline

Translation

The target sequence has its own length and order.

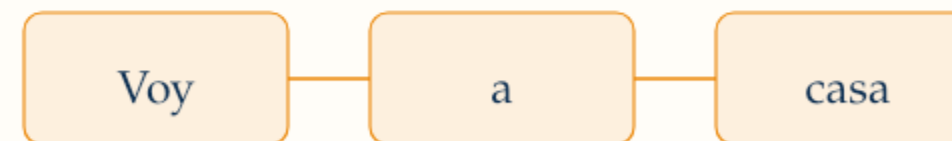
SOURCE $X_{1:n}$



$n \neq m$

Words may change position or combine across languages.

TARGET $Y_{1:m}$



The model must first represent the source, then generate the target on a separate recurrent timeline.

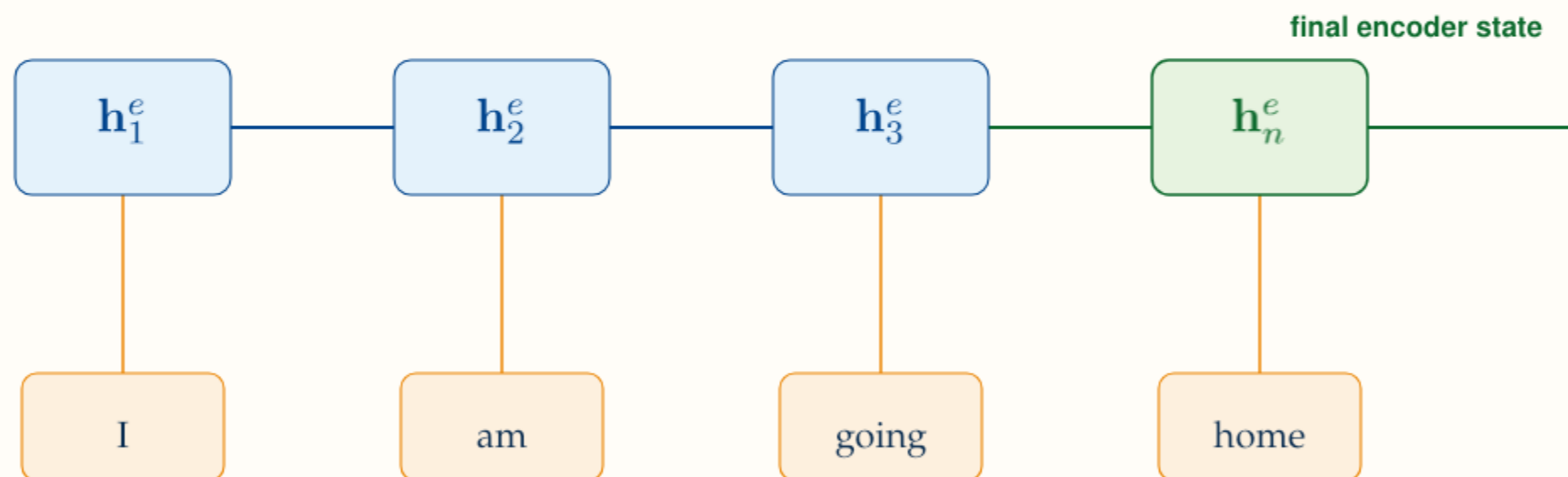
The Encoder Summarizes an Input Sequence

A recurrent encoder converts the source tokens into contextual hidden states.

ENCODING THE SOURCE SEQUENCE

The encoder creates one state at every source position

$$\mathbf{h}_i^e = f_{\text{enc}}(\mathbf{h}_{i-1}^e, E(x_i))$$



The final state carries the source history accumulated by the recurrent encoder.

Context Vector

Course notation

$$\mathbf{c}_{\text{ctx}} = \mathbf{h}_n^e$$

Jurafsky–Martin notation: c

A fixed-length summary
of the complete source
sequence

Encoder cell

Vanilla RNN, GRU or LSTM

A bidirectional encoder may combine two
terminal states.

The context must now condition a second recurrent computation.

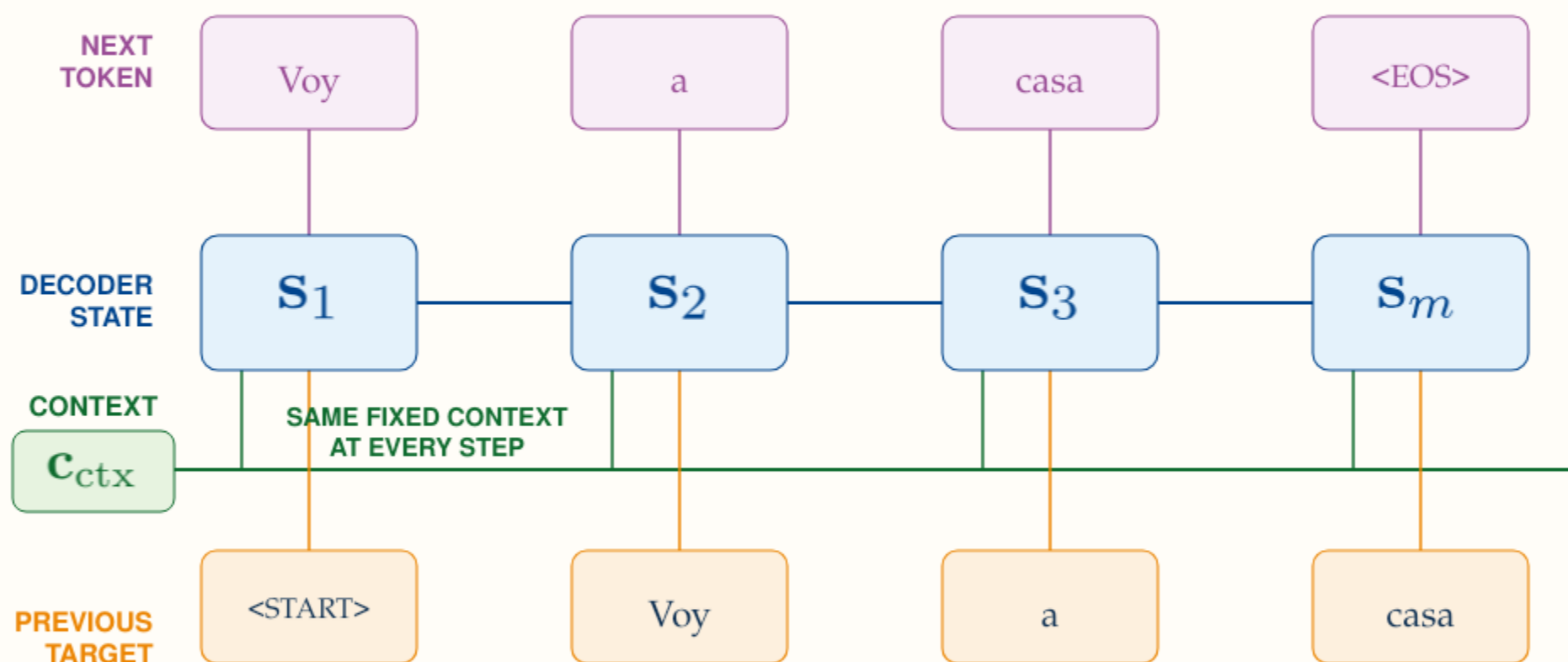
Sutskever et al. (2014)

The Decoder Generates One Output at a Time

Each decoder state uses the previous target token and the encoder context.

AUTOREGRESSIVE TARGET GENERATION

The target sequence has its own recurrent timeline



The output at one step becomes the previous-token input at the next step.

One Decoder Step

State update

$$s_t = g_{dec}(s_{t-1}, E(y_{t-1}), c_{ctx})$$

Vocabulary distribution

$$p(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(W_o s_t + \mathbf{b}_o)$$

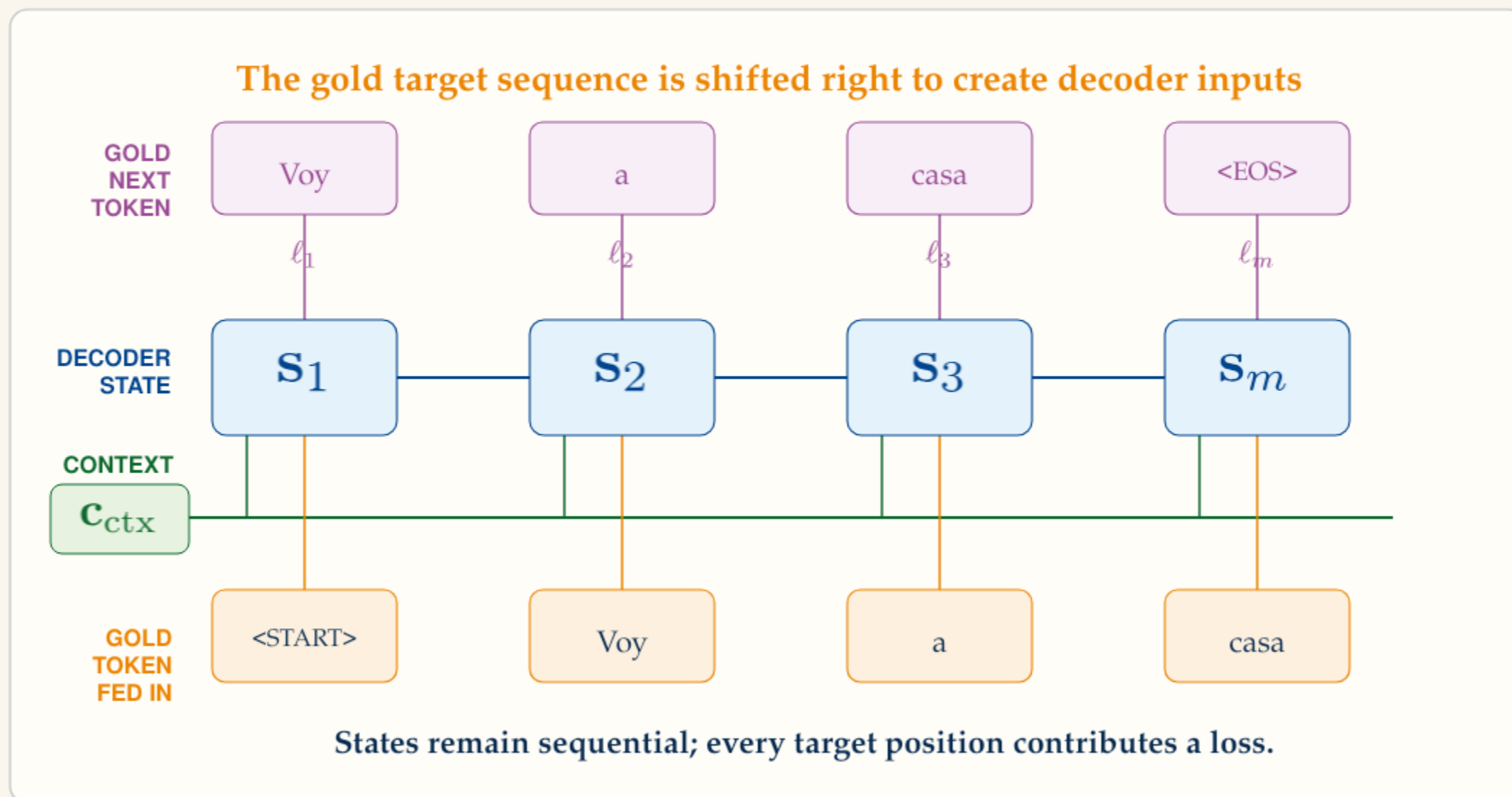
<START> begins generation.
<EOS> ends the target sequence.

During training, which previous token should enter the next decoder step?

Teacher Forcing During Training

The observed previous target token enters the decoder at every training step.

TRAINING THE RECURRENT DECODER



One Training Step

Teacher-forced state

$$s_t = g_{dec}(s_{t-1}, E(y_{t-1}^*), c_{ctx})$$

Token loss

$$\ell_t = -\log p(y_t^* | y_{<t}^*, \mathbf{x})$$

The decoder receives the correct previous token, even when its own prediction was wrong.

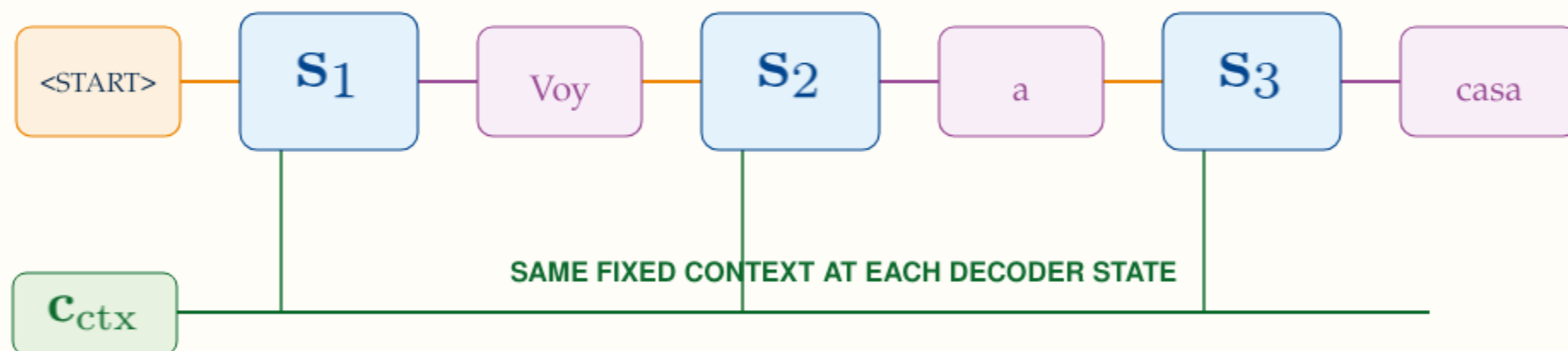
At inference time, the observed target sequence is unavailable.

Autoregressive Inference Uses Its Own Predictions

After <START>, each selected token becomes input to the following decoder step.

GENERATION WITHOUT A GOLD TARGET

Each selected token becomes part of the next decoder input



Only the model's selected tokens are available after <START>.
An early mistake changes the later decoder states.

One Inference Step

State update

$$\mathbf{s}_t = g_{\text{dec}}(\mathbf{s}_{t-1}, E(\hat{y}_{t-1}), \mathbf{c}_{\text{ctx}})$$

Vocabulary distribution

$$\mathbf{p}_t = \text{softmax}(\mathbf{W}_o \mathbf{s}_t + \mathbf{b}_o)$$

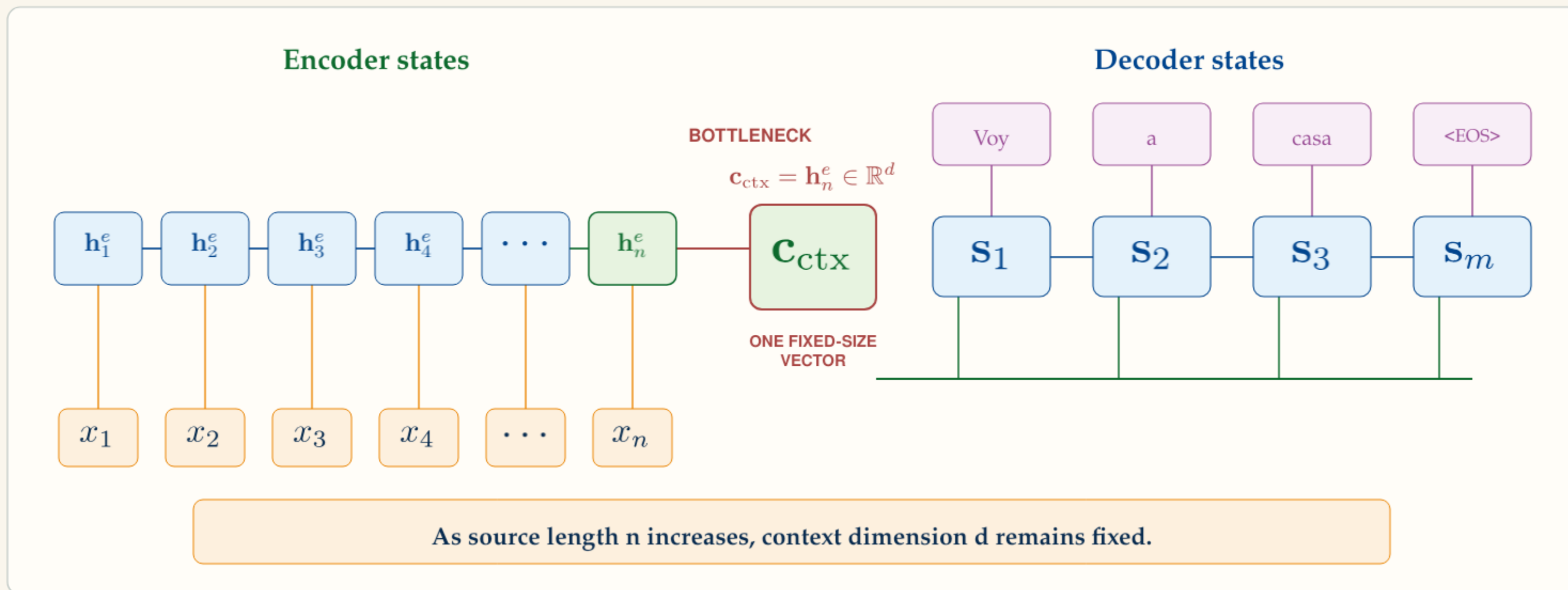
Select one token from the distribution.
<EOS> terminates generation.

The decoder can use only the source information contained in the fixed context.

The Fixed Context Bottleneck

Every source detail must pass through one fixed-length summary before decoding.

THE LIMIT OF FIXED-CONTEXT ENCODER-DECODERS



How does this bottleneck affect a translation whose target uses a different order?

Translation Can Reorder the Same Information

A natural Hindi sentence may arrange its phrases differently from the English source.

USE CASE: ENGLISH TO HINDI TRANSLATION

ENGLISH SOURCE

A
The doctor

B
prescribed

C
a new medicine

D
after examining

E
the old report

Colour links phrases that carry corresponding information.

English order: A B C D E

Hindi order: E D A C B

HINDI TARGET

E
पुरानी रिपोर्ट की

D
जाँच करने के बाद

A
डॉक्टर ने

C
एक नई दवा

B
लिखी

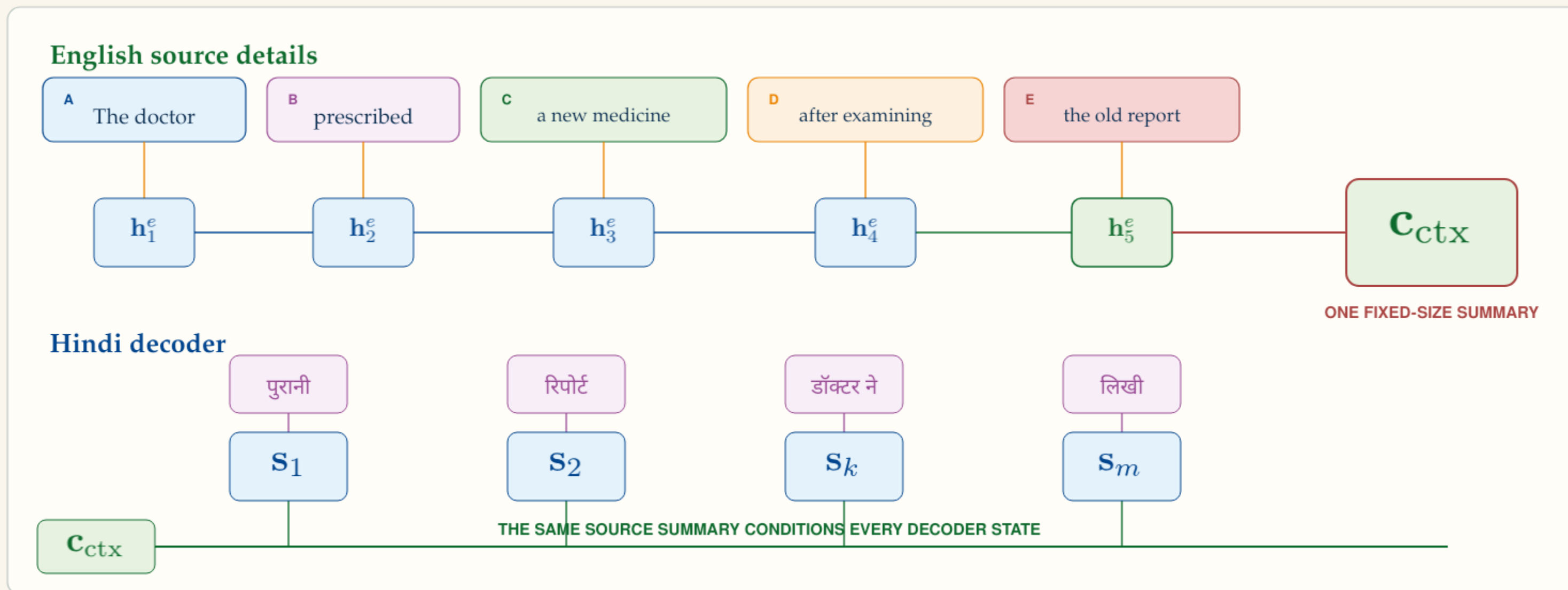
The Hindi sentence begins with information found at the end of the English source.

Translation requires the model to remember the content while choosing a new order.

A Fixed Context Must Preserve the Whole English Sentence

Before Hindi generation begins, the encoder compresses every source detail into one vector.

THE MEMORY BURDEN IN FIXED-CONTEXT TRANSLATION



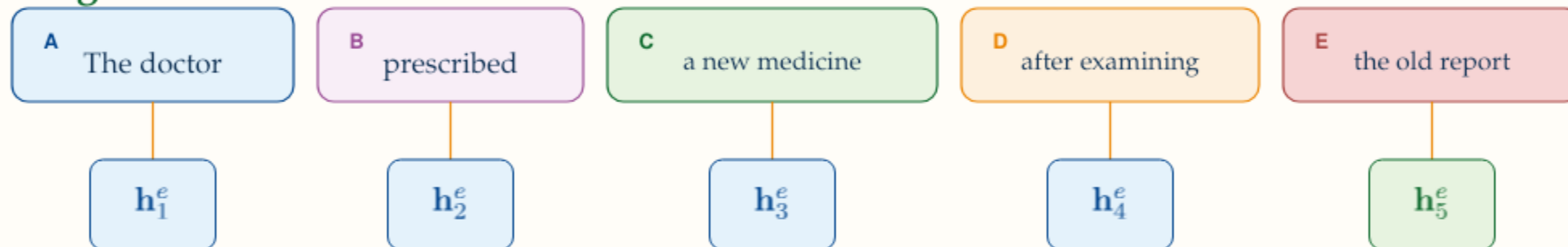
At each Hindi step, the decoder must recover a different source detail from the same vector.

Without Attention, the Decoder Cannot Revisit the Source

Each Hindi output step must recover the needed detail from the fixed context and its recurrent history.

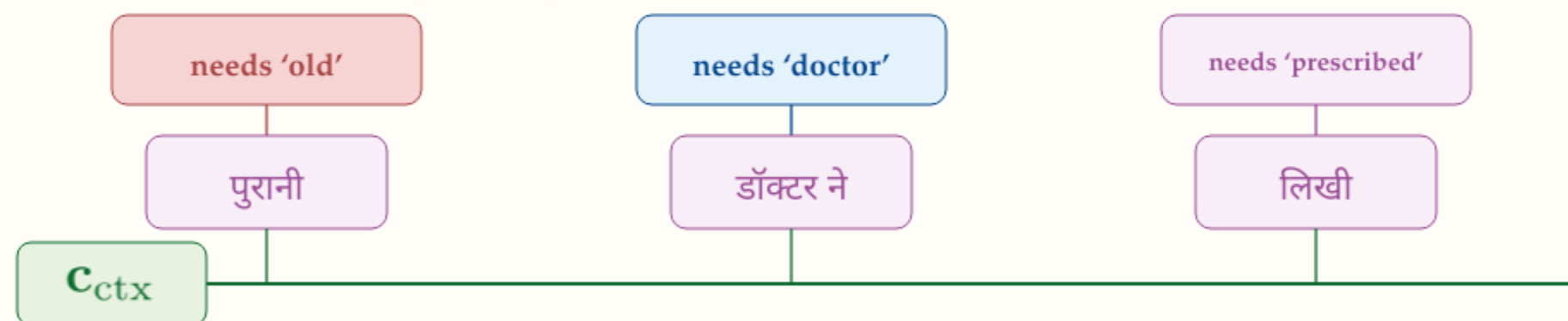
WHAT IS MISSING FROM THE DECODER INPUT?

English encoder states



NO DIRECT PATH FROM THESE STATES TO THE DECODER

Selected Hindi output steps



Decoder Inputs

$$\mathbf{s}_t = g_{\text{dec}}(\mathbf{s}_{t-1}, E(y_{t-1}), \mathbf{c}_{\text{ctx}})$$

Previous decoder state

Previous Hindi token

Fixed source context

No individual encoder state appears in this update.

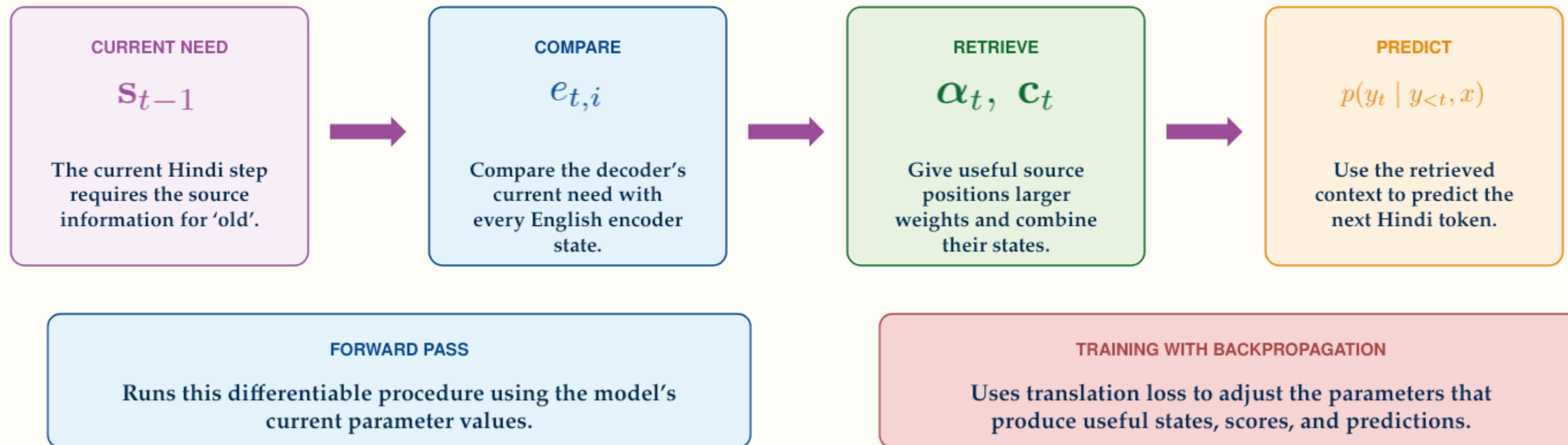
Attention gives each decoder step a weighted view of all encoder states.

Attention as a Retrieval Procedure

The forward pass turns the retrieval intuition into a computation that training can improve.

THE OVERALL IDEA

At each Hindi output step, the decoder runs the same four-part retrieval procedure.



The attention weights are recalculated for every sentence and every decoder step.

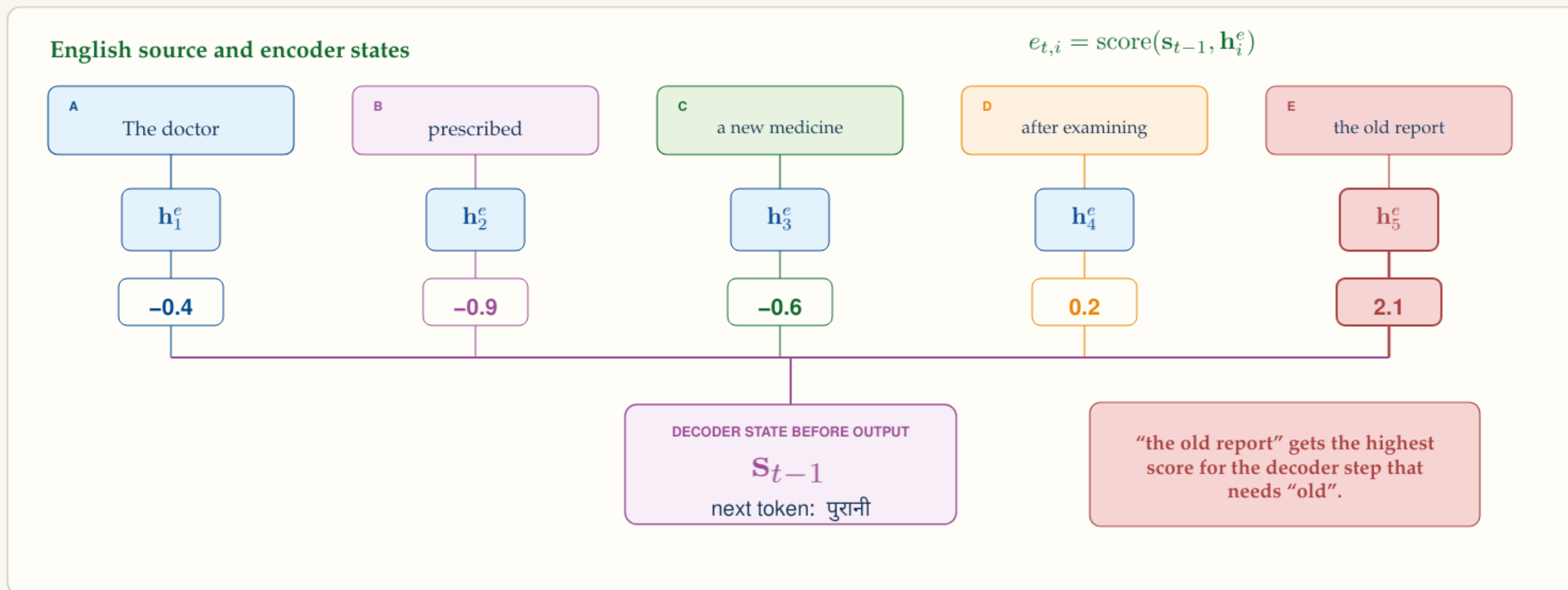
We now construct the procedure, beginning with the relevance scores.

Bahdanau et al. (2015)

Attention Scores Every Source State

Before generating the Hindi word for “old”, the decoder compares its current need with each English encoder state.

STEP 1: COMPUTE RELEVANCE SCORES

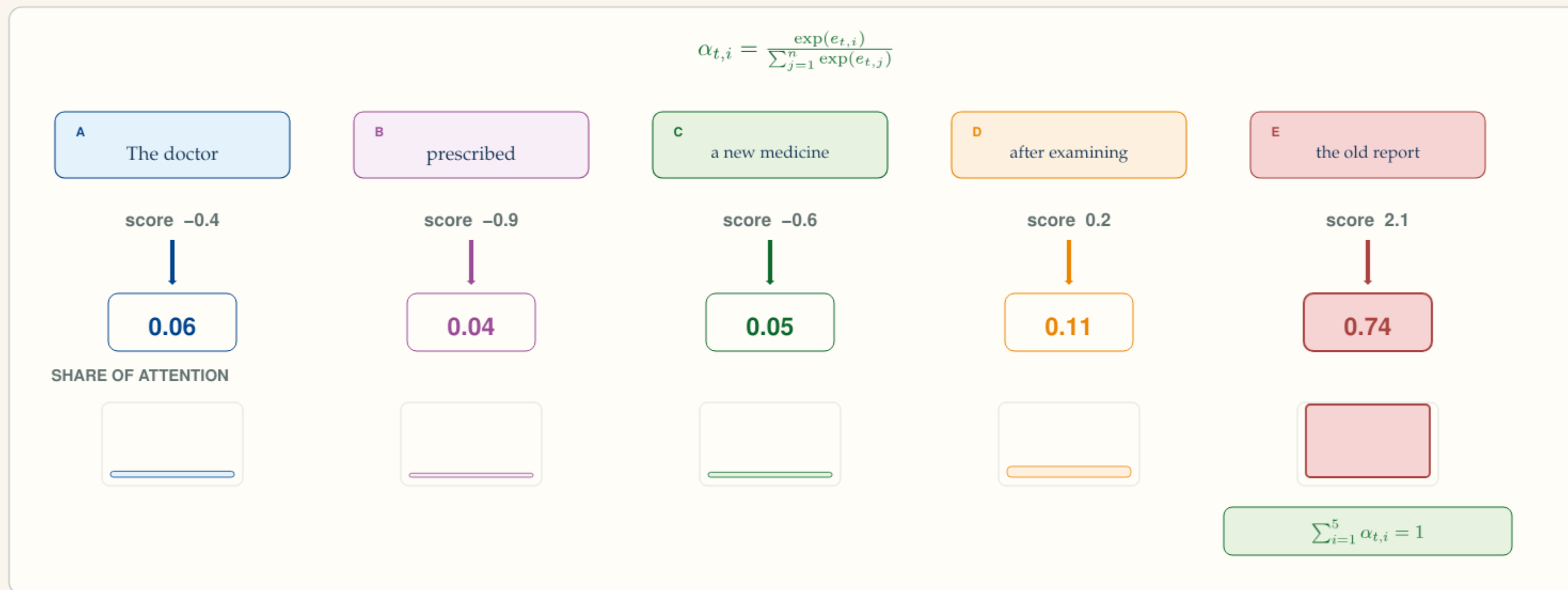


The scores may be positive or negative. Softmax converts them into attention weights.

Softmax Converts Relevance Scores into Attention Weights

The weights are positive and sum to one, so they quantify how strongly this output step uses each source state.

STEP 2: NORMALIZE THE SCORES

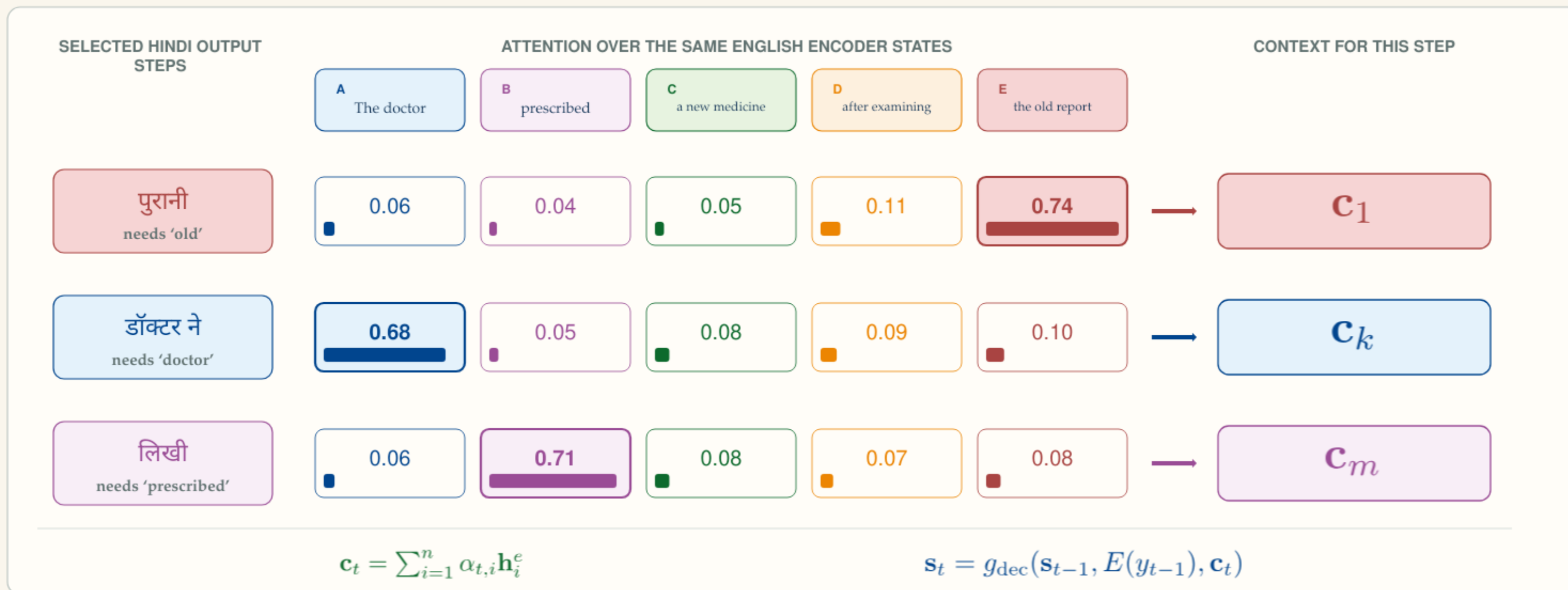


All five states contribute, with the largest share coming from “the old report”.

Each Hindi Output Step Gets Its Own Context Vector

Attention recomputes the weights as the decoder moves through the reordered target sentence.

STEP 3: BUILD A DYNAMIC SOURCE SUMMARY

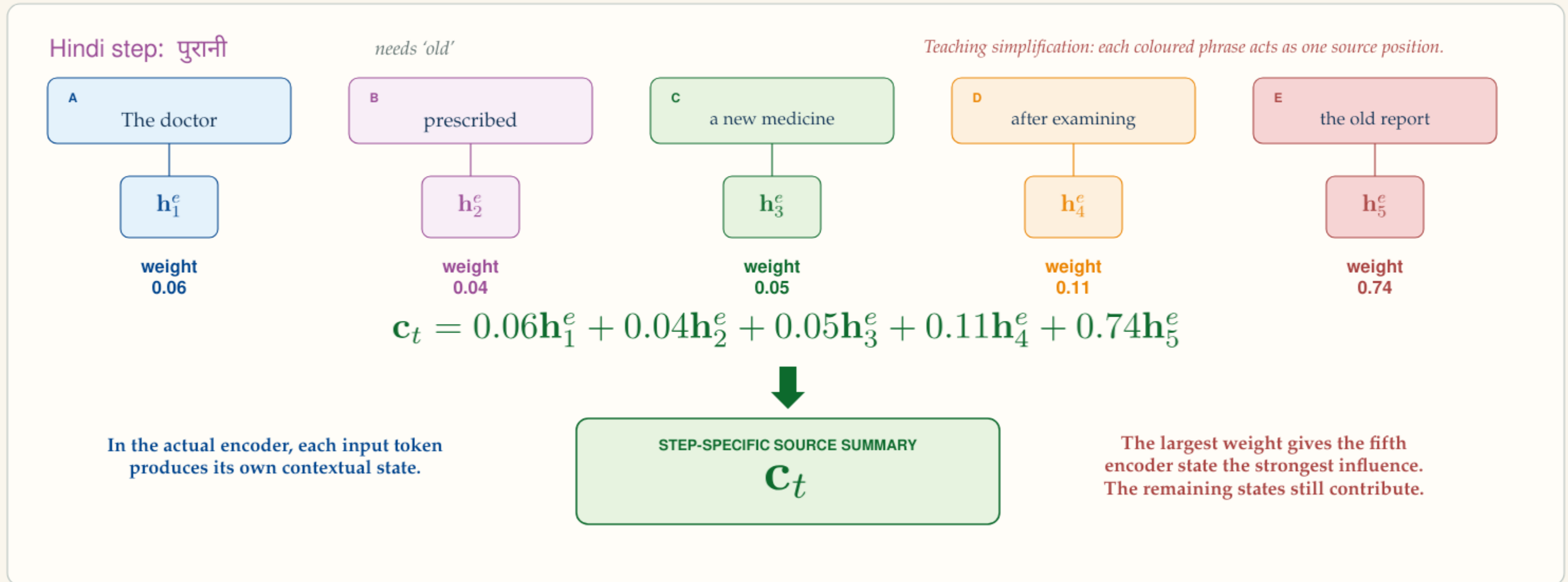


The decoder can revisit the English information needed at each Hindi output step.

Context Is a Weighted Sum

Each encoder state contributes in proportion to its attention weight for the current Hindi output step.

UNPACKING THE DYNAMIC CONTEXT

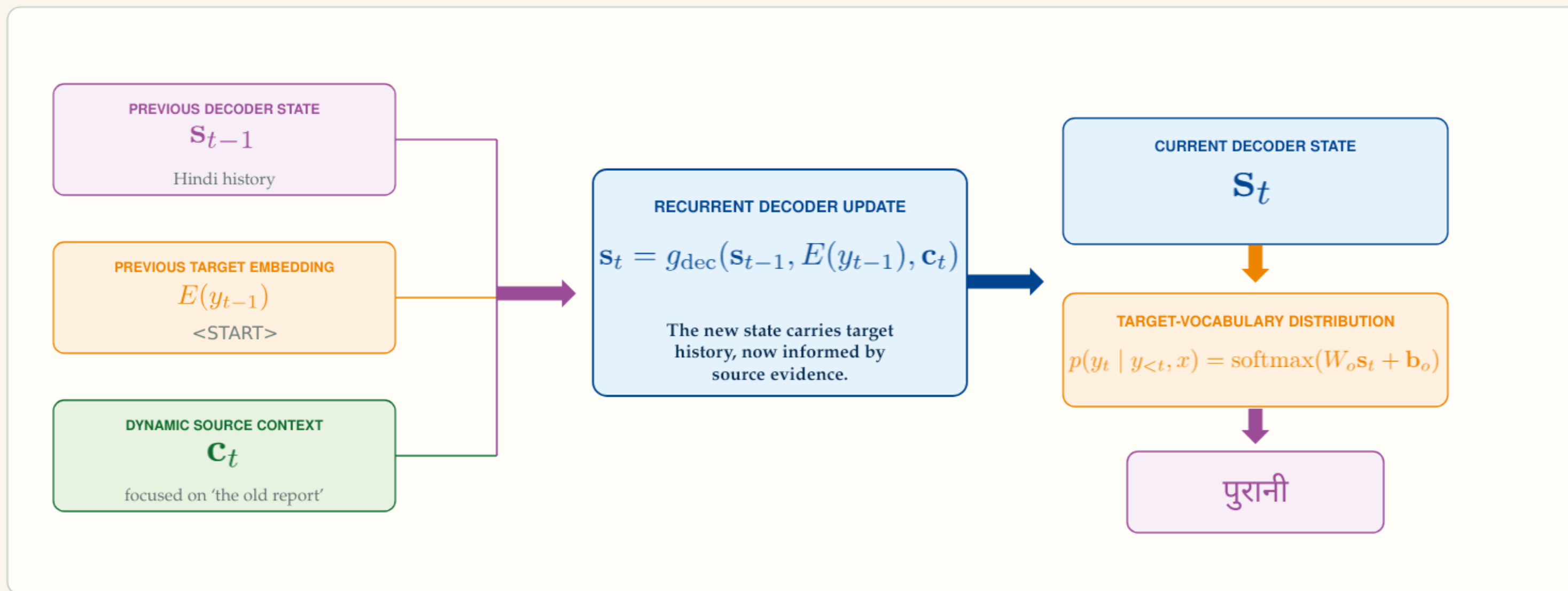


The decoder now combines this source summary with its recurrent target history.

Context Conditions the Decoder

The decoder combines source evidence with its recurrent target history before predicting the next Hindi token.

USING THE DYNAMIC CONTEXT



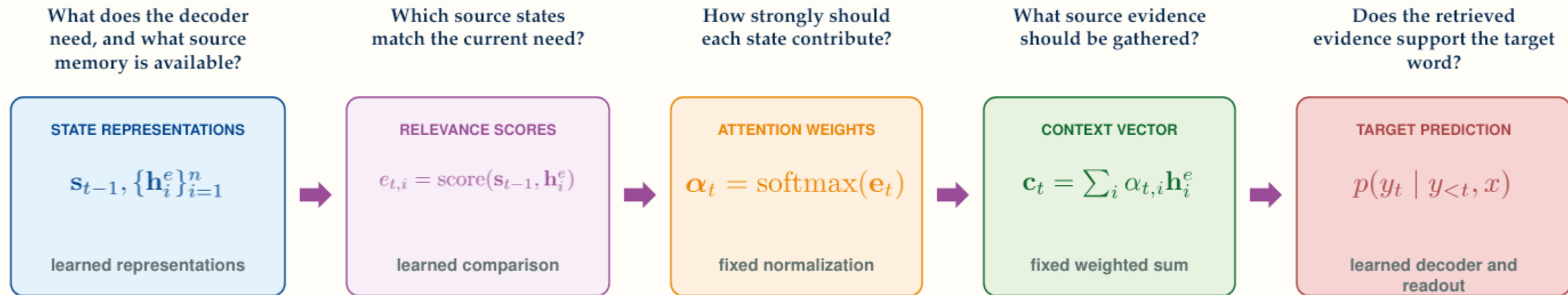
The forward pass now contains a complete rule for retrieving and using source information.

Attention as a Trainable Algorithm

The forward pass encodes a retrieval strategy using the model's current parameter values.

THE ATTENTION HYPOTHESIS

With suitable parameter values, this computation can recover the source information needed at each target step.



The architecture makes selective retrieval a precise, differentiable computation. Its numerical parameters determine whether the computation is useful.

The attention weights are recomputed for every sentence and every decoder step.

Translation loss can now test whether this retrieval procedure helped predict the correct word.

Architecture, Forward Pass, and Training

The architecture expresses a retrieval hypothesis. Learning finds parameter values that make it useful.

SEPARATING THE PROCEDURE FROM ITS NUMBERS

THE DESIGNED PROCEDURE

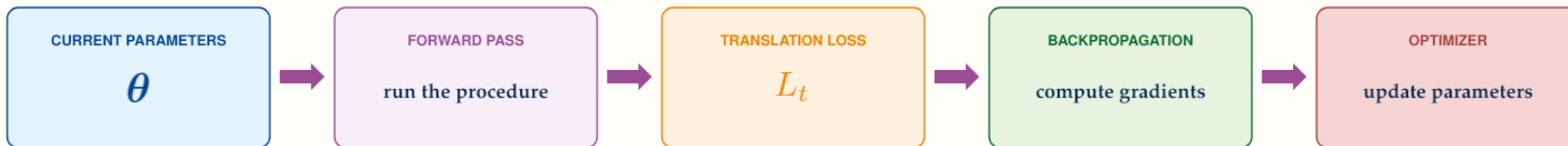
We choose a differentiable sequence of operations that could implement selective retrieval.



This structure states how a decoder may retrieve useful source information.

THE TRAINING LOOP

Training runs the procedure with current parameters. The resulting error guides the next update.



$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

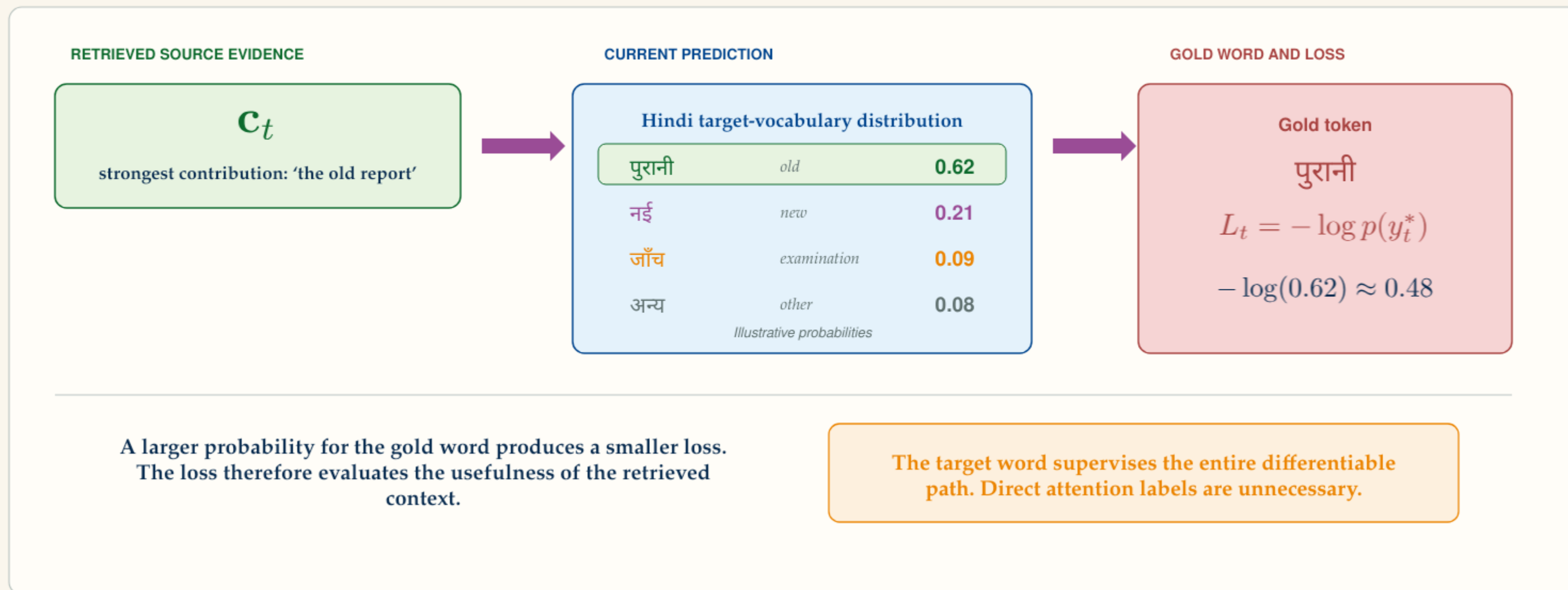
Backpropagation computes the gradient. The optimizer uses it to change the parameters.

The gold translation supplies the error signal that judges the retrieval procedure.

Translation Loss Evaluates the Retrieved Context

The gold Hindi token tests whether the retrieved source evidence supported the correct prediction.

ONE DECODER STEP DURING TRAINING



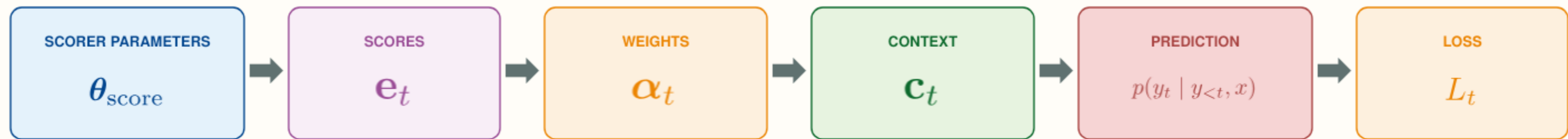
Differentiability allows the loss to reach the scoring parameters that shaped the context.

Backpropagation Through Attention

The loss traces responsibility backward through prediction, context, weights, and relevance scores.

HOW TRANSLATION ERROR CHANGES RETRIEVAL

The forward computation created a path from the scoring parameters to the translation loss.



Backpropagation follows this same path in reverse and applies the chain rule.



gradient information flows backward

$$\frac{\partial L_t}{\partial \theta_{\text{score}}} = \frac{\partial L_t}{\partial c_t} \frac{\partial c_t}{\partial \alpha_t} \frac{\partial \alpha_t}{\partial e_t} \frac{\partial e_t}{\partial \theta_{\text{score}}}$$

The gradient measures how a small parameter change would alter the loss through every intervening operation.

The same loss also updates the encoder, decoder, embeddings, and output readout.

Repeated updates make the scoring rule useful across many sentences and target positions.

Learned Parameters and Computed Attention Weights

Training stores a shared scoring rule. Each decoder step uses that rule to compute a fresh pattern of attention.

WHAT THE MODEL LEARNS AND WHAT THE FORWARD PASS CREATES

PERSIST ACROSS EXAMPLES

encoder and decoder weights

token embeddings

scoring parameters such as W_s

output readout parameters

STORED AFTER TRAINING

RECOMPUTED FOR EACH STEP

encoder states h_i^e

relevance scores $e_{t,i}$

attention weights $\alpha_{t,i}$

context vector c_t

DEPEND ON x AND s_{t-1}

ONE SCORING RULE

$$e_{t,i} = s_{t-1}^\top W_s h_i^e$$

$e_{t,i}$ is computed

W_s is learned

The same learned rule can produce different attention patterns for different sentences.

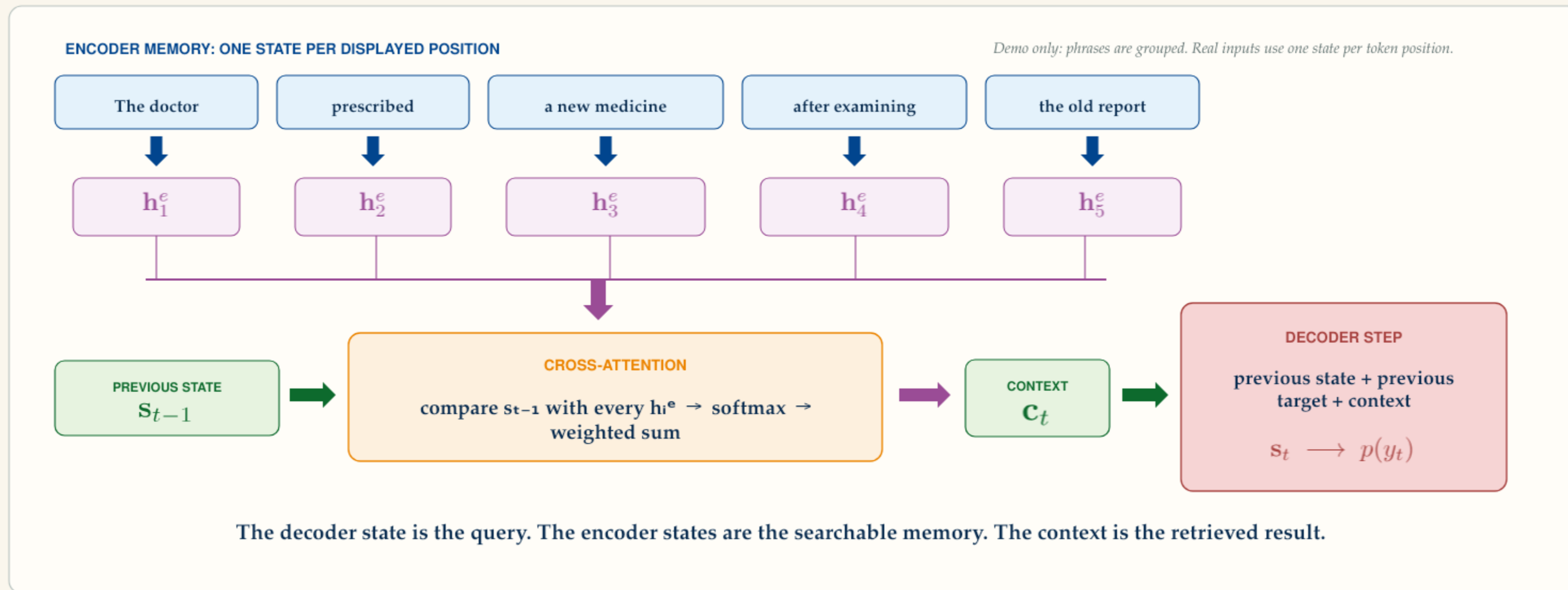
The learned parameters now operate inside the complete encoder-decoder system.

Luong, Pham & Manning (2015)

The Complete Attentional Encoder–Decoder

At every target step, the decoder queries all source states, retrieves a context, and predicts the next token.

PUTTING THE PIECES BACK TOGETHER



Repeating this computation for every target step produces a complete attention matrix.

Attention as Soft Alignment

Across target steps, attention traces which parts of the English sentence are most useful for producing each Hindi phrase.

THE RETRIEVAL PATTERN ACROSS A COMPLETE TRANSLATION

HINDI OUTPUT STEPS

ENGLISH SOURCE POSITIONS

	The doctor	prescribed	a new medicine	after examining	the old report
डॉक्टर ने	0.78	0.05	0.04	0.05	0.08
पुरानी रिपोर्ट	0.04	0.04	0.05	0.10	0.77
की जाँच करने के बाद	0.03	0.05	0.05	0.76	0.11
नई दवा	0.05	0.06	0.76	0.07	0.06
लिखी	0.08	0.73	0.08	0.06	0.05

HOW TO READ THE MATRIX

Each row is one attention distribution.

Dark cells show the strongest retrieved source evidence.

The diagonal can bend because English and Hindi use different word orders.

Translation loss can produce this pattern without word-alignment labels.

डॉक्टर ने पुरानी रिपोर्ट की जाँच करने के बाद नई दवा लिखी।

Illustrative attention weights. They show model routing, not a guaranteed human explanation.

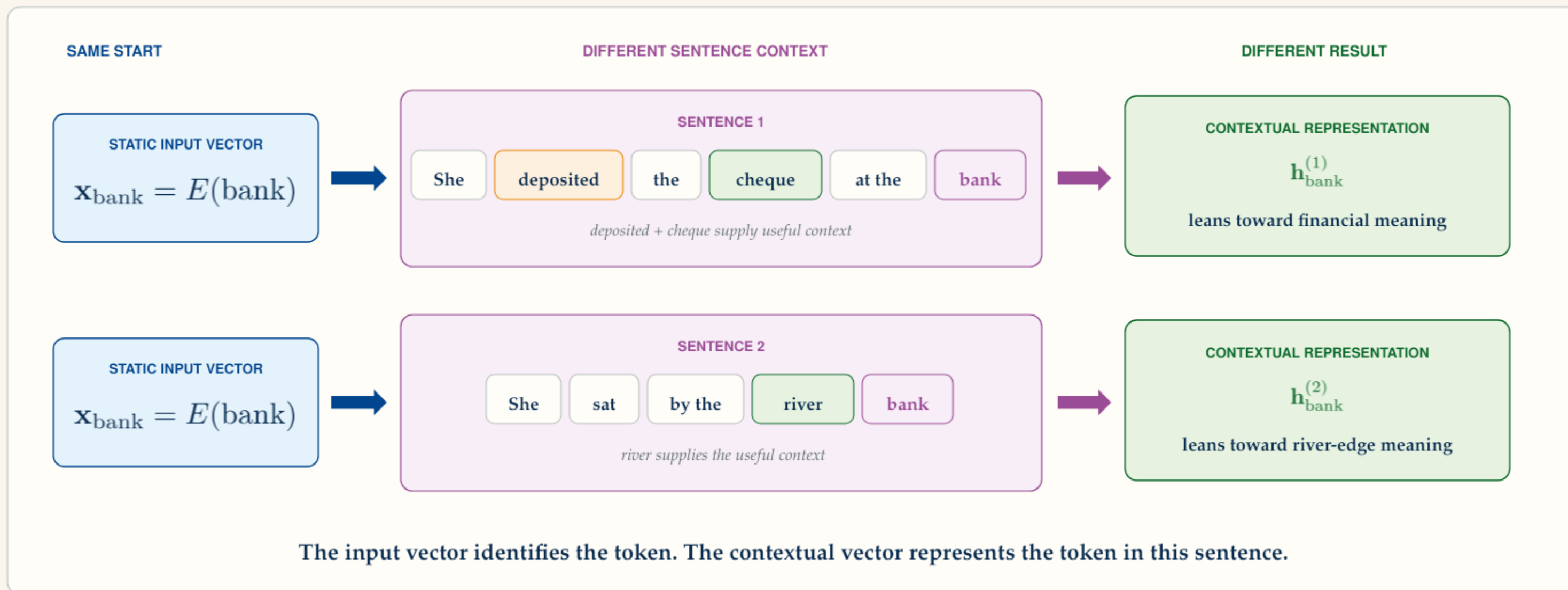
The next question: can attention construct the representations as well as retrieve them?

Bahdanau et al. (2015)

Why Token Representations Must Depend on Context

A static embedding supplies a starting point. Attention builds a sentence-specific representation.

THE SAME TOKEN CAN PLAY A DIFFERENT ROLE IN A DIFFERENT SENTENCE



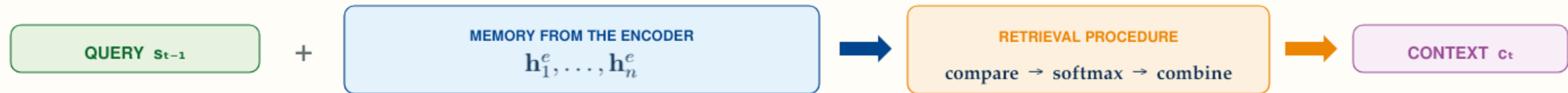
How can one token retrieve the contextual information it needs from its own sequence?

Cross-Attention and Self-Attention

The retrieval procedure stays the same. The source of the query and memory changes.

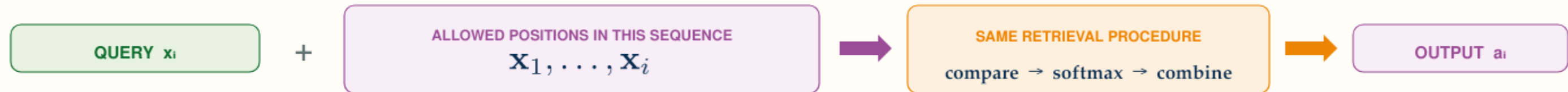
SAME ALGORITHM, DIFFERENT PARTICIPANTS

CROSS-ATTENTION



A decoder state retrieves source information from a different sequence.

SELF-ATTENTION



Each position retrieves information from other allowed positions in the same sequence.

Causal attention sees the current and earlier positions.
A bidirectional encoder may attend to positions on both sides.

The Transformer makes the comparison and retrieval roles explicit through queries, keys, and values.